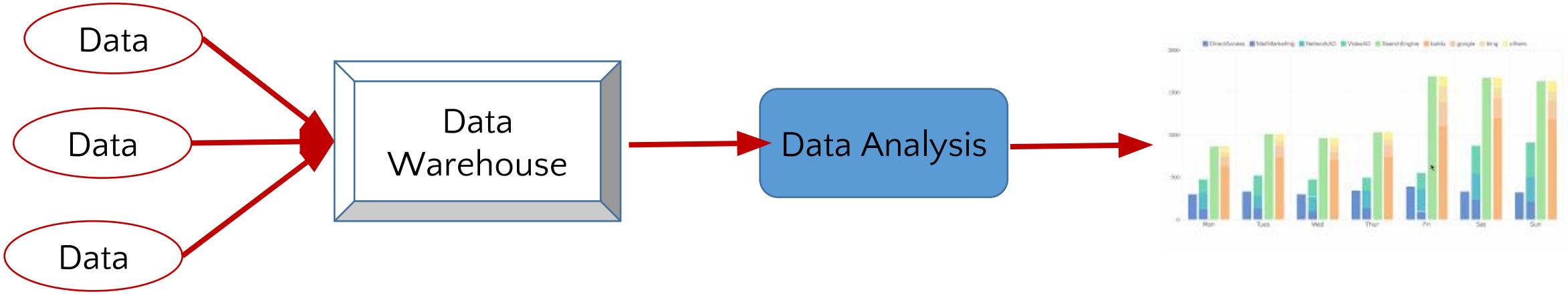
python

Data science using python

Introduction

- Data plays very important role in our lives.
- Every piece of data is important for any business organization for decision making.
- So we need a mechanism to store these data, organize data and the analyze the data so top level management takes some decision
- The person who analyze the data are known as **data analyst**.
- The job of data analyst is retrieve data as per requirement from central storage, arrange and analyze the data and find some knowledge.
- Based on this knowledge, top level management takes appropriate decision.



Introduction

- Once data is analyzed, it is presented to end user in the form of pictures or graphs so that end used can easily understand
- This presentation is known as visualization
- The main goal of data visualization is to communicate information clearly and efficiently using statistical graphs, plots and diagrams
- Data science is term used to extract information from the data warehouse, analyze them and present the necessary data to business organization in order to take appropriate decision
- A person who involved in this work is known as **data scientist**.

Introduction

- The difference between data analyst and data scientist is as

Data analyst	Data scientist
Data analyst receives questions from the business team and provides answer to then	Data scientist formulate the questions that will help a business organization and proceed in solving them
Simply analyze the data and provide information requested by the team	Will have strong data visualization skills and the ability to convert data into business story
Perfect in data warehouse, big data concept, SQL and business intelligence is needed for data analyst	Perfect in mathematical, statistical and programming language like R and python
Looks at known data from new perspectives	Estimates the unknown information from the known data
a data analyst makes sense out of existing data	a data scientist works on new ways of capturing and analyzing data to be used by the analysts.
a data analyst is someone who merely curates meaningful insights from data	A data scientist is someone who can predict the future based on past patterns

Data Structures in panda

- There are two data structures in panda

1.DataFrame

2.Series

Python Pandas Series

- **Pandas Series is a one-dimensional labeled array capable of holding data of any type (integer, string, float, python objects, etc.).**

In the real world, a Pandas Series will be created by loading the datasets from existing storage, storage can be SQL Database, CSV file, and Excel file.
Data of series is always mutable. Means it can be changed. But the size of data series is always immutable, means it can not be changed.
Pandas Series can be created from the lists, dictionary, and from a scalar value etc. Series can be created in different ways, here are some ways by which we create a series: syntax

<series object> =pandas.Series(data,index=idx(optional))

Creating a series from array: In order to create a series from array, we have to import a numpy module and have to use array() function.

```
# import pandas as pd
import pandas as pd

# import numpy as np
import numpy as np

# simple array
data = np.array(['g','e','e','k','s'])

ser = pd.Series(data)
print(ser)
```

Python Pandas Series

Creating a series from array with an index: In order to create a series by explicitly proving index instead of the default, we have to provide a list of elements to the index parameter with the same number of elements as it is an array.

```
# import pandas as pd
import pandas as pd

# import numpy as np
import numpy as np

# simple array
data = np.array(['g','e','e','k','s'])

ser = pd.Series(data, index=[12,13,14,15,16])
print(ser)
```

Accessing element of Series:

There are two ways through which we can access element of series, they are :

1. Accessing Element from Series with Position

In order to access the series element refers to the index number. Use the index operator [ ] to access an element in a series. The index must be an integer. In order to access multiple elements from a series, we use Slice operation.

```
import pandas as pd
data = {'g':10,'e':20,'e':30,'k':40,'s':50}
ser = pd.Series(data)
print(ser[:2])
```

Accessing element of Series:

There are two ways through which we can access element of series, they are :

2. Accessing Element Using Label (index)

In order to access an element from series, we have to set values by index label. A Series is like a fixed-size dictionary in that you can get and set values by index label.

```
# import pandas as pd
import pandas as pd

# import numpy as np
import numpy as np

# simple array
data = np.array(['g','e','e','k','s'])

ser = pd.Series(data, index=[12,13,14,15,16])
print(ser)
```

Python Pandas Series

Creating a series from Dictionary:

- In order to create a series from the dictionary, we have to first create a dictionary after that we can make a series using dictionary.
- Dictionary keys are used to construct indexes of Series.

```
 import pandas as pd
import pandas as pd
data = {'g':10,'e':20,'e':30,'k':40,'s':50}
ser = pd.Series(data)
print(ser)
```

Data Frame

- A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. OR
- Data Frame is an object that is useful in representing data in the form of rows and columns
- Features of DataFrame
  - Potentially columns are of different types
  - Size – Mutable
  - Labeled axes (rows and columns)
  - Can Perform Arithmetic operations on rows and columns
- The data may comes from different sources like excel spreadsheet, python sequence like list or tuple
- Such data can be represented in the form of data frame which is further useful for various operations for analyzing and understanding of data

Data Frame

- Data frame are generally create from .csv (comma separated value) file, Excel spreadsheet files, python dictionaries, list, tuple or list of dictionaries.
- Python contains ***pandas*** module which is useful for data analysis and manipulations.
- Also ***xlrd*** is a package that is useful to retrieve data from excel
- DataFrame is the main object in pandas package
- To install these two packages following commands are used
  - ***pip install pandas***
  - ***Pip install xlrd***

Create Data Frame

pandas.DataFrame
A pandas DataFrame can be created using the following constructor –

pandas.DataFrame(data, index, columns, dtype, copy)

Sr.No	Parameter & Description
1	**data** data takes various forms like ndarray, series, map, lists, dict, constants and also another DataFrame.
2	**index** For the row labels, the Index to be used for the resulting frame is Optional Default np.arange(n) if no index is passed.
3	**columns** For column labels, the optional default syntax is - np.arange(n). This is only true if no index is passed.
4	**dtype** Data type of each column.
5	**copy** This command (or whatever it is) is used for copying of data, if the default is False.

Create Data Frame

```
import pandas as pd

data = {'product_name': ['laptop', 'printer', 'tablet', 'desk', 'chair'],
        'price': [1200, 150, 300, 450, 200]     }

df = pd.DataFrame(data)

print (df)
```

Create Data Frame

```
import pandas as pd

data = {'product_name': ['laptop', 'printer', 'tablet', 'desk', 'chair'],
        'price': [1200, 150, 300, 450, 200]      }

df = pd.DataFrame(data,index=['product_1','product_2','product_3','product_4','product_5'])

print (df)
```

Create Data Frame

```
import pandas as pd

data = [[‘rahul', 10], [‘Anil', 15], [‘raju', 14]]

# Create the pandas DataFrame
df = pd.DataFrame(data, columns = ['Name', 'Age'])

print(df)
```

Create Data Frame

- To create data frame from pandas then first import pandas module as
- *import pandas as pd*
- **Empty Data Frame**
- *df = pd.DataFrame()*
- **DataFrame from Lists**
- *data = [1,2,3,4,5]*
- *df = pd.DataFrame(data)*
- *data = [['Alex',10],['Bob',12],['Clarke',13]]*
- *df = pd.DataFrame(data,columns=['Name','Age'])*

- **DataFrame from Lists**
- *data = [['Alex',10],['Bob',12],['Clarke',13]]*
- *df = pd.DataFrame(data,columns=['Name','Age'],dtype=float)*

pandas.DataFrame

A pandas DataFrame can be created using the following constructor –

pandas.DataFrame(data, index, columns, dtype, copy)

Create Data Frame from Excel Spreadsheet

- First create excel file with extension .xlsx
- To create data frame, we should first import pandas package and also xlrd package which is useful in extracting data from Excel data.
- To read the data from excel file, we should used read_excel() function of pandas package and the syntax is as
- ***read_excel('file path','sheet number')***
- When we create data frame from excel, first column shows index column starting from 0.

```
import pandas as pd
df=pd.read_excel("c:\\users\\a\\student.xlsx","Sheet1")
```

Create Data Frame from CSV file

- CSV is comma separator value file which is similar to Excel file but it takes less memory.
- We can create csv file by saving Excel file using the option File->Save As and then Choose save as type csv
- To read the data from csv file, we should used read_csv() function of pandas package and the syntax is as
- ***read_excel('file path')***
- When we create data frame from excel, first column shows index column starting from 0.

```
import pandas as pd
df=pd.read_excel("c:\\users\\a\\student.csv")
```

Create Data Frame from Python Dictionary

- In some situation data was available in python dictionary format.
- Python dictionary contains key-value pairs.
- In this situation we need to convert python dictionary into data frame
- This can be achieved by passing dictionary to DataFrame class object as
- ***Df=pd.DataFrame(dictionary_Name)***

```
empdata={"empno":[1001,1002,1003,1004,1005,1006],"ename":["Ajit","Vijay","Sunil","Rani","Rekha","Satish"
], "address":["Sangola","Sangola","Pune","Sangli","Satara","Sangola"],
“sal":[12000,16000,30000,19000,25000,36000],
"dob":["10-Jun-1999","15-Apr-1998","01-Dec-1999","20-Sep-2000","30-Oct-1995","22-Feb-1994"] }

import pandas as pd
df=pd.DataFrame(empdata)
```

Create Data Frame from List of Tuples

- In some situation data was available in python's List of Tuple format.
- In this case tuple can be treated as row of data.
- In this situation we need to convert List of Tuple into data frame
- This can be achieved by passing List of Tuple to DataFrame class object as
- ***Df=pd.DataFrame(list_of_tuple_name)***

```
emp=[(1001,"Ajit","Sangola",12000,"10-Jan-1999"),(1002,"Vijay","Sangola",16000,"15-Apr-1998"),(1003,"Sunil","Pune",3
0000,"01-Dec-1999"), (1004,"Rani","Sangli",19000,"20-Sep-2000"),
(1005,"Rekha","Satara",25000,"30-Oct-1995"),(1006,"Satish","Sangola",36000,"22-Feb-1994")]


import pandas as pd
df=pd.DataFrame(emp,columns=("Empno","Ename","Address","Sal","Birth date"))
```

- Since original list dose not has column names, so by using ***columns*** option we can add column name

Operations on DataFrame

- Once we create a data frame, we can do various operations on it.
- These operations are useful in analyzing the data or manipulating the data
- **Importing Data**
- Use these commands to import data from a variety of different sources and formats.

Method	Description
pd.read_csv(filename)	From a CSV file
pd.read_table(filename)	From a delimited text file (like TSV)
pd.read_excel(filename)	From an Excel file
pd.read_sql(query, connection_object)	Read from a SQL table/database
pd.read_json(json_string)	Read from a JSON formatted string, URL or file.
pd.read_html(url)	Parses an html URL, string or file and extracts tables to a list of dataframes
pd.read_clipboard()	Takes the contents of your clipboard and passes it to read_table()
pd.DataFrame(dict)	From a dict, keys for columns names, values for data as lists

Operations on DataFrame

- **Exporting Data**
- Use these commands to export a DataFrame to CSV, .xlsx, SQL, or JSON.

Method	Description
df.to_csv(filename)	Write to a CSV file
df.to_excel(filename)	Write to an Excel file
df.to_sql(table_name, connection_object)	Write to a SQL table
df.to_json(filename)	Write to a file in JSON format

- **Create Test Objects**
- These commands can be useful for creating test segments.

Method	Description
pd.DataFrame(np.random.rand(20,5))	5 columns and 20 rows of random floats
pd.Series(my_list)	Create a series from an iterable my_list
df.index = pd.date_range('1900/1/30', periods=df.shape[0])	Add a date index

Operations on DataFrame

- **Viewing/Inspecting Data**
- Use these commands to take a look at specific sections of your pandas DataFrame or Series.

df.head(n)	First n rows of the DataFrame
df.tail(n)	Last n rows of the DataFrame
df.shape	Number of rows and columns
df.info()	Index, Datatype and Memory information
df.describe()	Summary statistics for numerical columns
s.value_counts(dropna=False)	View unique values and counts
df.apply(pd.Series.value_counts)	Unique values and counts for all columns

- **Selection**
- Use these commands to select a specific subset of your data.

df[col]	Returns column with label col as Series
df[[col1, col2]]	Returns columns as a new DataFrame
s.iloc[0]	Selection by position
s.loc['index_one']	Selection by index
df.iloc[0,:]	First row
df.iloc[0,0]	First element of first column

Operations on DataFrame

- **Data Cleaning**
- Use these commands to perform a variety of data cleaning tasks.

df.columns = ['a','b','c']	Rename columns
pd.isnull()	Checks for null Values, Returns Boolean Arrray
pd.notnull()	Opposite of pd.isnull()
df.dropna()	Drop all rows that contain null values
df.dropna(axis=1)	Drop all columns that contain null values
df.dropna(axis=1,thresh=n)	Drop all rows have have less than n non null values
df.fillna(x)	Replace all null values with x
s.fillna(s.mean())	Replace all null values with the mean (mean can be replaced with almost any function from the statistics module)
s.astype(float)	Convert the datatype of the series to float
s.replace(1,'one')	Replace all values equal to 1 with 'one'
s.replace([1,3],['one','three'])	Replace all 1 with 'one' and 3 with 'three'
df.rename(columns=lambda x: x + 1)	Mass renaming of columns
df.rename(columns={'old_name': 'new_ name'})	Selective renaming
df.set_index('column_one')	Change the index
df.rename(index=lambda x: x + 1)	Mass renaming of index

Operations on DataFrame

- **Filter, Sort, and Groupby**
- Use these commands to filter, sort, and group your data.

df[df[col] > 0.5]	Rows where the column col is greater than 0.5
df[(df[col] > 0.5) & (df[col] < 0.7)]	Rows where 0.7 > col > 0.5
df.sort_values(col1)	Sort values by col1 in ascending order
df.sort_values(col2,ascending=False)	Sort values by col2 in descending order
df.sort_values([col1,col2],ascending=[True,False])	Sort values by col1 in ascending order then col2 in descending order
df.groupby(col)	Returns a groupby object for values from one column
df.groupby([col1,col2])	Returns groupby object for values from multiple columns
df.groupby(col1)[col2]	Returns the mean of the values in col2, grouped by the values in col1 (mean can be replaced with almost any function from the statistics module)
df.pivot_table(index=col1,values=[col2,col3],aggfunc=mean)	Create a pivot table that groups by col1 and calculates the mean of col2 and col3
df.groupby(col1).agg(np.mean)	Find the average across all columns for every unique col1 group
df.apply(np.mean)	Apply the function np.mean() across each column
nf.apply(np.max,axis=1)	Apply the function np.max() across each row

Operations on DataFrame

- **Join/Combine**
- Use these commands to combine multiple dataframes into a single one.

df1.append(df2)	Add the rows in df1 to the end of df2 (columns should be identical)
pd.concat([df1, df2],axis=1)	Add the columns in df1 to the end of df2 (rows should be identical)
df1.join(df2,on=col1,how='inner')	SQL-style join the columns in df1 with the columns on df2 where the rows for col have identical values. 'how' can be one of 'left', 'right', 'outer', 'inner'

- **Statistics**
- Use these commands to perform various statistical tests. (These can all be applied to a series as well.)

df.describe()	Summary statistics for numerical columns
df.mean()	Returns the mean of all columns
df.corr()	Returns the correlation between columns in a DataFrame
df.count()	Returns the number of non-null values in each DataFrame column
df.max()	Returns the highest value in each column
df.min()	Returns the lowest value in each column
df.median()	Returns the median of each column
df.std()	Returns the standard deviation of each column

Python Pandas Series

- **Pandas Series is a one-dimensional labeled array capable of holding data of any type (integer, string, float, python objects, etc.).**

```
# import pandas as pd
import pandas as pd
# simple array
data = [1, 2, 3, 4]
ser = pd.Series(data)
print(ser)
```

python

Introduction to Numpy

Introduction

- NumPy is a Python package, which stands for Numerical Python.
- NumPy contains multidimensional array objects and the routines which process these arrays.
- The package was created by Travis Oliphant.
- It has almost all the features of Numeric, its predecessor, and Numarray.
- One of the greatest advantages of NumPy is its ability to perform operations similar to those in MATLAB. Note that NumPy is open source as against MATLAB.
- The package can be easily installed ON your computer from htpps://www.continuum.io.
- You can download Anaconda, which is a free distribution software for SciPy and related packages.
- Using NumPy, multidimensional arrays can be subjected to mathematical and logical operations, transforms like Fourier Transforms and operators related to linear algebra and random numbers.
- The ability to manipulate multidimensional arrays assumes importance as they are used in various algorithms including those of machine learning.

Introduction

- The ndarray is the most important object in NumPy, as it helps to create an array.
- These arrays are zero based indexed.
- The elements of the array are assigned consecutive memory locations.
- These elements are then arranged in the row major style (as in the case of C) or column major style (as in the case of FORTRAN).
- In Python a basic array can be created using the ***numpy.array*** function.
- The function takes the following arguments:
  - ***Object:*** It returns an array or nested sequence
  - ***dtype:*** The data type of the array is represented by dtype
  - ***copy:*** The object is copied if the value of this argument is true
  - ***order:*** An array can be row major (C) or column major (F) or any (A)
  - ***subok:*** Returned array forced is to be a base class array
  - ***ndmin:*** This argument specifies the minimum number of dimensions

Introduction

1. import numpy as np
2. array1 = np.array([7, 14, 21, 28, 35])
3. array1

- ***Output*** array([7, 14, 21, 28, 35])

1. the NumPy package is imported as 'np'.
2. The array function of np is then used to create an array ([7, 14, 21, 28, 35]).
3. Finally, the value of the array1 is displayed

- Two Dimensional Array
- array2=np.array([[1,2,3],[4,5,6],[7,8,9]])

Introduction

- An array of string type can be created by specifying the datatype (dtype = str) in the array function of np.
- array2=np.array([[1,2,3],[4,5,6],[7,8,9]], dtype=str)
- It creates array o string values

Introduction

- NumPy is the fundamental package for scientific computing with Python.
- It stands for “Numerical Python.”
- It supports:
  - N-dimensional array object
  - Broadcasting functions
  - Tools for integrating C/C++ and Fortran code
  - Useful linear algebra, Fourier transform, and random number capabilities
- Besides its obvious scientific uses, NumPy can also be used as a multi-dimensional container to store generic data.
- Arbitrary data types can also be defined.
- This allows NumPy to seamlessly and speedily integrate with a wide variety of databases.
- NumPy’s main object is the homogeneous multidimensional array.

Why?

- One of the most obvious questions is why the need to use these arrays in place of those provided by Python?
- The first reason is the wide variety of data types provided by NumPy.
- The variety of data types provided by the package makes the task at hand easy.
- Some of the important data types provided by NumPy are as follows:
  - bool_: Boolean
  - int_: default integer type
  - intc: Same as that of 'C'
  - NumPy also provides int8, int16, int32, and int64 data types.
  - The corresponding unsigned integers are unit8, uint16, uint32, and uint64.
- The difference between int and uint is that in the former, one bit is reserved for sign and in the latter all the bits are used for storing the numbers, and hence the unsigned integers can store larger values.

Why?

- The data types for floating point numbers are as follow:
  - float_
  - float16: It has 10 bit mantissa
  - float32: It has 23 bit mantissa
  - flaot64: It has 52 bit mantissa
- The data types for complex numbers are as follows:
  - complex_
  - complex64
  - Complex128

Example

1. import numpy as np
2. int_number_array = np.array([1,2,3,4])
3. int_number_array
   - array([1, 2, 3, 4])
4. type(int_number_array)
   - <class 'numpy.ndarray'>
5. int_number_array.dtype
   - dtype('int32')

1. Import the numpy library into your program
2. Pass a list of items to the np.array() function and assign the result to int_number_array object.
3. In the output, you can see all the elements are placed within an iterable object of array class and is a one-dimensional array. The int_number_array object belongs to numpy.ndarray class .
4. NumPy provides a large set of numeric datatypes that you can use to construct arrays. NumPy tries to guess a datatype when you create an array, but functions that construct arrays also usually include an optional argument to explicitly specify the datatype.
5. The type of int_number_array is dtype('int32').

Example

1. float_number_array = np.array([9.1, 8.1, 8.8, 3.0])
2. float_number_array.dtype
   - dtype('float64')
3. two_dimensional_array_list = np.array([[1,2,3], [4,5,6]])
4. two_dimensional_array_list
   - array([[1, 2, 3],
   - [4, 5, 6]])
5. two_dimensional_array_tuple = np.array(((1,2,3), (4,5,6)))
6. two_dimensional_array_tuple
   - array([[1, 2, 3],
   - [4, 5, 6]])

- The type of float_number_array is dtype('float64').
- The np.array() function takes either single list or tuple as an argument. If you want to specify multiple lists or tuples, then pass it as nested lists or nested tuples
- Here, two_dimensional_ array_list and two_dimensional_array_tuple are examples for two-dimensional arrays.

Example

```
1. array_dtype = np.array([1,2,3,4], dtype = np.float64)
2. array_dtype
    1. array([1., 2., 3., 4.])
3. array_dtype.dtype
    1. dtype('float64')
```

- You can also explicitly specify the data type of array by assigning type values like np.float64, np.int32, and others to a dtype attribute and pass it as a second argument to np.array() function.

Array Attribute

- The contents of ndarray can be accessed and modified by indexing or slicing the array and via the methods and attributes of the ndarray.
- The more important attributes of ndarray object are

Array Attribute	Description
ndarray.ndim	Gives the number of axes or dimensions in the array
ndarray.shape	Gives the dimensions of the array. For an array with n rows and m columns, shape will be a tuple of integers (n, m).
ndarray.size	Gives the total number of elements of the array.
ndarray.dtype	Gives an object describing the type of the elements in the array. One can create or specify dtype's using standard Python types. Additionally, NumPy provides its own types like np.int32, np.int16, np.float64, and others.
ndarray.itemsize	Gives the size of each element of the array in bytes.
ndarray.data	Gives the buffer containing the actual elements of the array. Normally, we will not use this attribute because we will access the elements in an array using indexing facilities.

Array Attribute

1. import numpy as np
2. array_attributes = np.array([[10, 20, 30], [14, 12, 16]])
3. array_attributes.ndim
   - 2
4. array_attributes.shape
   - (2, 3)
5. array_attributes.size
   - 6
6. array_attributes.dtype
   - dtype('int32')
7. array_attributes.itemsize
   - 4
8. array_attributes.data
   - <memory at 0x000001E61DB963A8>

NumPy Arrays Creation

- NumPy Arrays Creation with Initial Placeholder Content
  - the elements of an array are initially unknown, but its size is known.
  - Hence, NumPy offers several functions to create arrays with initial placeholder content
  - These minimize the necessity of growing arrays, an expensive operation.

Function Name	Description
np.zeros()	Creates an array of zeros
np.ones()	Creates an array of ones
np.empty()	Creates an empty array
np.full()	Creates a full array
np.eye()	Creates an identity matrix
np.random.random()	Creates an array with random values
np.arange()	The syntax for arange() is, np.arange([start,]stop, [step,][dtype=None]) Returns evenly spaced values within a given interval where start (a number and optional) is the start of interval and its default value is zero, stop (a number) is the end of interval, and step (a number and is optional) is the spacing between the values and dtype is the type of output array.

NumPy Arrays Creation

1. import numpy as np
2. np.zeros((2,3))
   - array([[0., 0., 0.],
   - [0., 0., 0.]])
3. np.ones((3,4))
   - array([[1., 1., 1., 1.],
   - [1., 1., 1., 1.],
   - [1., 1., 1., 1.]])
4. np.empty((2,3))
   - array([[0., 0., 0.],
   - [0., 0., 0.]])
5. np.full((3,3),2)
   - array([[2, 2, 2],
   - [2, 2, 2],
   - [2, 2, 2]])

1. np.eye(2,2)
   - array([[1., 0.],
   - [0., 1.]])
2. np.random.random((2,2))
   - array([[0.95022839, 0.23253555],
   - [0.843828 , 0.57976282]])
3. np.arange(10, 30, 5)
   - array([10, 15, 20, 25])
4. np.arange(0, 2, 0.3)
   - array([0. , 0.3, 0.6, 0.9, 1.2, 1.5, 1.8])
5. np.linspace(0, 2, 9)
   - array([0. , 0.25, 0.5 , 0.75, 1. , 1.25, 1.5 , 1.75, 2.])

NumPy Arrays Creation

- When the arange() function is used with floating point arguments, it is generally not possible to predict the number of elements obtained, due to the finite floating point precision.
- For this reason, it is usually better to use the linspace() function to which you can pass an argument specifying the number of elements you want to generate instead of the step.
- In , the linspace() function produces nine data elements between zero and two.
- The ***logspace*** function generates a sequence which is equally spaced in the logspace. That is the elements of the sequence will be between basestart and basestop "," where the value of the base is provided in the argument.
- The default value of base is 10.
- The dtype signifies the data type of the elements.
- Like in the case of the previous function, endpoint= False would exclude basestop NumPy.
- logspace (start, stop, num, endpoint, base, dtype)

Aggregate Functions

- The numpy module contains many aggregate functions.
- The various functions along with a brief explanation are as follows:

Aggregate Functions	Descriptions
numpy.sum	This finds the sum of the elements of the argument (e.g. a list or an array)
numpy.prod	This finds the product of the elements of the argument (e.g. a list or an array)
numpy.mean	This finds the mean of the elements of the argument (e.g. a list or an array)
numpy.std	This finds the standard deviation of the elements of the argument
numpy.var	This finds the variance of the elements of the argument
numpy.argmin	This finds the position (index) of the maximum element
numpy.argmax	This finds the position of the minimum element
numpy.median	This finds the median of the elements of the argument
numpy.any	This finds if any element of the given argument is there
numpy.all	This finds if all the arguments of the given argument are there

Aggregate Functions

- The numpy module contains many aggregate functions.
- The various functions along with a brief explanation are as follows:

Aggregate Functions	Descriptions
numpy.max	This finds the maximum element of the argument. In the case of a list or a 1D array, the maximum element will be displayed. However, in the case of a 2D array the axis along which the maximum element is desired can also be mentioned. Here, axis=0 indicates columns and axis=1 indicates rows.
numpy.min	This finds the minimum element of the argument. In the case of a list or a 1D array, the minimum element will be displayed. However, in the case of a 2D array the axis along which the minimum element is desired can also be mentioned. Here, axis=0 indicates columns and axis=1 indicates rows.

Aggregate Functions

```
import numpy as np
B=100*(np.random.random(50))
Max=np.max(B)
Max_Index=np.argmax(B)
Min=np.min(B)
Min_Index=np.argmin(B)
Sum=np.sum(B)
Prod=np.prod(B)
Mean=np.mean(B)
SD=np.std(B)
Variance=np.var(B)
Med=np.median(B)
Per25=np.percentile(B,25)
Per75=np.percentile(B,75)
```

```
print("Max\t:",Max,"\nIndex\t:",Max_Index,"\nMin\t:",Min,"\nIndex\t:",Min_Index,"\nAverage\t:",Mean,"\nStadDeviation\t:",SD,"\nVariance\t:",Variance,"\nMedian\t:",Med,"\nPercentile 25\t:",Per25,"\nPercentile75\t:",Per75)
```

```
Max: 99.93710854682874
Index      : 31
Min : 2.33727937956415
Index      : 24
Average : 52.97451916520782
StadDeviation      : 29.21277232720687
Variance: 853.3860670412236
Median  : 50.107605212729
Percentile 25: 30.431378383719355
Percentile75 : 78.15458384866659
```

Array Indexing, Slicing and Iterating

- One-dimensional arrays can be indexed, sliced, and iterated over, much like lists and other Python sequences
- For multi-dimensional arrays you can specify an index or slice per axis.

- a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
- a[1, 3]
  - 8
- a[:2, 1:3]
  - array([[2, 3],
  - [6, 7]])
- lower_axes = a[1, :]
- lower_axes
  - array([5, 6, 7, 8])
- lower_axes.ndim
  - 1

- same_axes = a[1:2, :]
- same_axes
  - array([[5, 6, 7, 8]])
- same_axes.ndim
  - 2
- a[:, 1]
  - array([2, 6, 10])
- a[:, 1:2]
  - array([[2],
  - [6],
  - [10]])

- for row in a:
- ... print(row)
  - [1 2 3 4]
  - [5 6 7 8]
  - [9 10 11 12]
- for each_element in a.flat:
- ... print(each_element)
  - Print 1 to 12

Array Indexing, Slicing and Iterating

- if one wants to perform an operation on each element in the array, one can use the flat attribute, which is an iterator over all the elements of the array
- You can construct an array from another array

- a = np.array([[1, 2], [3, 4], [5, 6]])
- a[[0, 1, 2], [0, 1, 0]]
  - array([1, 4, 5])
  - Here Elements found in (0, 0), (1, 1), and (2, 0) index are pulled out and displayed

- Boolean array indexing lets you select the elements of an array that satisfy some condition.
- a= np.array([[11, 12], [13, 14], [15, 16]])
- a[a > 13]
  - array([14, 15, 16])
  - a[a > 13] , elements greater than 13 in the array a are displayed as a one-dimensional array.

Basic Arithmetic Operations on NumPy Arrays

- Basic mathematical functions perform element-wise operation on arrays and are available both as operator overloads and as functions in the NumPy module.

- a = np.array([20, 30, 40, 50])
- b = np.arange(4)
- b
  - array([0, 1, 2, 3])
- a + b
  - array([20, 31, 42, 53])
- np.add(a, b)
  - array([20, 31, 42, 53])
- a – b
  - array([20, 29, 38, 47])
- np.subtract(a, b)
  - array([20, 29, 38, 47])

- A = np.array([[1, 1], [6, 1]])
- B = np.array([[2, 8], [3, 4]])
- A * B
  - array([[2, 8],
  - [18, 4]])
- np.multiply(A, B)
  - array([[2, 8],
  - [18, 4]])
- A / B
  - array([[0.5 , 0.125],
  - [2. , 0.25]])

- np.divide(A, B)
  - array([[0.5 , 0.125],
  - [2. , 0.25]])
- np.dot(A, B)
  - array([[5, 12],
  - [15, 52]])
- B**2
  - array([[4, 64],
  - [9, 16]], dtype=int32)

Mathematical Functions in NumPy

- Various mathematical functions are supported in NumPy.
- A few frequently used mathematical functions are shown below

- a = np.array([20, 30, 40, 50])
- np.sin(a)
  - array([0.91294525, -0.98803162, 0.74511316, -0.26237485])
- np.cos(a)
- np.tan(a)
- a = np.array([-1.7, -1.5, -0.2, 0.2, 1.5, 1.7, 2.0])
- np.floor(a)
  - array([-2., -2., -1., 0., 1., 1., 2.])
- np.ceil(a)
  - array([-1., -1., -0., 1., 2., 2., 2.])

- The sum() function returns the sum of array elements over a given axis

- np.sqrt([1,4,9])
  - array([1., 2., 3.])
- np.maximum([2, 3, 4], [1, 5, 2])
  - array([2, 5, 4])
- np.minimum([2, 3, 4], [1, 5, 2])
  - array([1, 3, 2])
- np.sum([0.5, 1.5])
  - 2.0
- np.sum([[0, 1], [0, 5]], axis=0)
  - array([0, 6])
- np.sum([[0, 1], [0, 5]], axis=1)
  - array([1, 5])

Changing the Shape of an Array

- You can change the shape of an array.

- a = np.floor(10*np.random.random((3,4)))
- a
  - array([[2., 3., 2., 5.],
  - [3., 3., 8., 7.],
  - [8., 6., 5., 0.]])
- a.shape
  - (3, 4)
- a.ravel()
  - array([2., 3., 2., 5., 3., 3., 8., 7., 8., 6., 5., 0.])
- a.reshape(6,2)
  - array([[2., 3.],
  - [2., 5.],
  - [3., 3.],
  - [8., 7.],
  - [8., 6.],
  - [5., 0.]])

- The shape of an array can be changed with ravel() and reshape() functions.
- Both ravel() and reshape() return a modified array but do not change the original array.
- The function ravel() returns a flattened array, such as a one-dimensional array, containing the elements of the input.
- The function reshape() gives a new shape to an array without changing its data.

Stacking and Splitting of Arrays

- You can stack several arrays together or split an array to several arrays.

- a = np.array([[3, 1], [8, 7]])
- b = np.array([[2, 4], [4, 8]])
- np.vstack((a, b))
  - array([[3, 1],
  - [8, 7],
  - [2, 4],
  - [4, 8]])
- np.hstack((a, b))
  - array([[3, 1, 2, 4],
  - [8, 7, 4, 8]])

- Several arrays can be stacked together along different dimensions using vstack() and hstack() functions.
- With vstack() and hstack() functions you are stacking the arrays a and b together in row-wise and column-wise fashion.
- The number of columns when stacking with vstack() and the number of rows when stacking with hstack() should be the same.
- Use hsplit() function to split an array along its horizontal axis.

Stacking and Splitting of Arrays

- You can stack several arrays together or split an array to several arrays.

- a = np.floor(10*np.random.random((2, 12)))
- a
  - array([[8., 3., 6., 3., 5., 5., 5., 8., 9., 7., 6., 8.],
  - [8., 3., 1., 2., 9., 0., 5., 5., 0., 3., 3., 8.]])
- np.hsplit(a, 3)
  - [array([[8., 3., 6., 3.],
  - [8., 3., 1., 2.]]), array([[5., 5., 5., 8.],
  - [9., 0., 5., 5.]]), array([[9., 7., 6., 8.],
  - [0., 3., 3., 8.]])]
- np.hsplit(a, (3, 4))
  - [array([[8., 3., 6.],
  - [8., 3., 1.]]), array([[3.],
  - [2.]]), array([[5., 5., 5., 8., 9., 7., 6., 8.],
  - [9., 0., 5., 5., 0., 3., 3., 8.]])]
- np.vsplit(a, 2)
  - [array([[2., 9., 4., 4., 3., 0., 2., 9., 1., 2., 0., 1.]]), array([[3., 4., 8., 2., 5., 8., 5., 5., 7., 7.,7., 8.]])]

- You can either specify the number of equally shaped arrays to return or specify the columns after which the division should occur.
- Use vsplit() function to split an array along the vertical axis

python

Introduction to Matplotlib

Introduction

- It is important to be able to visualize the data as well.
- Visualization gives an insight of the results and may help to uncover the underlying patterns. ***Matplotlib*** is a package that helps to plot various types of graphs and visualize the data.
- Here we discusses the pyplot package of matplotlib.
- The pyplot collection of the matplotlib provides a set of functions which help programmers to perform various tasks associated with plotting.
- pyplot provides functions to plot a figure, create a plotting area, assign labels, etc.
- The pyplot keeps an account of the current figure and the area and hence can direct the functions to the requisite axes.

Data visualization

Data visualization is the most important step in the life cycle of data science

Univariate, Bivariate and Multivariate statistical analysis.

- Categorical variables — variables that have a finite number of categories or distinct groups. Examples: gender, method of payment, horoscope, etc.
- Numerical variables — variables that consist of numbers. There are two main numerical variables.
  - Discrete variables — variables that can be counted within a finite time. Examples: the change in your pocket, number of students in a class, numerical grades, etc.
  - Continuous variables — variables that are infinite in number often measured on a scale of sort. Examples: weight, height, temperature, date and time of a payment, etc.

Univariate data

- This type of data consists of **only one variable**. The analysis of univariate data is thus the simplest form of analysis since the information deals with only one quantity that changes.
- The most common univariate analysis is checking the central tendency (mean, median and mode), the range, the maximum and minimum values, and standard deviation of a variable.
- Common visual technique used for univariate analysis is a histogram, which is a frequency distribution graph. You could also use a box plot to compare the spread of the variables and provides an insight into outliers.

Heights (in cm)	**164**	**167.3**	**170**	**174.2**	**178**	**180**	**186**

Bivariate data

- This type of data involves **two different variables**. The analysis of this type of data deals with causes and relationships and the analysis is done to find out the relationship among the two variables.
- These variables could be dependent or independent to each other. In Bivariate analysis is that there is always a Y-value for each X-value.
- The most common visual technique for bivariate analysis is a scatter plot, where one variable is on the x-axis and the other on the y-axis.

TEMPERATURE(IN CELSIUS)	ICE CREAM SALES
20	2000
25	2500
35	5000
43	7800

Multivariate data

- When the data involves **three or more variables**, it is categorized under multivariate.
- For three variables, you can create a 3-D model to study the relationship (also known as Trivariate Analysis).
- Multivariate refers to multiple dependent variables that result in one outcome. This means that a majority of our real-world problems are multivariate. For example, based on the season, we cannot predict the weather of any given year. Several factors play an important role in predicting the same. Such as, humidity, precipitation, pollution, etc.
- Determining the value of an apartment. Factors possibly related to the value are size of the apartment, age of the building, number of bedrooms, number of bathrooms, and location (e.g. floor, view, etc.).

The Plot Function

- We start by plotting the values of a list.
- In order to generate a basic plot, a list L, having n values (index: 0 to (n-1)), can be passed to the plot function.
- In the generated plot, the x-axis will have values from 0 to (n-1) and the Y-axis will have the values in the given list.
- The *xlabel* function associates a label with the X-axis and which is a string.
- The *ylabel* function associates a label with the Y-axis.
- The *show* function displays the figure.
- One can also save the figure using the *savefig* function and it can takes two arguments: the figure to be plotted and the dpi.

The Plot Function

- import matplotlib.pyplot as plt
- plt.plot([1,4,8,10])
- plt.xlabel("X Axis")
- plt.ylabel("Y Axis")
- plt.show()
- plt.savefig("line.png",dpi=80)

- This can be changed using the axis function, which takes a list as an argument.
- The function has the following arguments:
- xmin
- ymin
- xmax
- ymax

- the list L =[1, 4, 8, 10] is passed to the plot function.
- The string "X Axis" is passed to the xlabel function and "Y Axis" is passed to the ylabel function.
- Note that in the figure, the X axis has values from 0 to 3 (the indices of the values of the list) and the Y axis ranges from 0 to 10 (from 0 to the maximum value of the list).
- If you provide a single list or array to plot, matplotlib assumes it is a sequence of y values, and automatically generates the x values for you.

The Plot Function

```
import matplotlib.pyplot as plt
X=[-5,-4,-3,-2,-1,0,1,2,3,4,5]
Y= [2*x*x-3 for x in X]
plt.plot(X,Y)
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
plt.savefig("line.png",dpi=80)
```

- the plot function takes one argument.
- However, it can also take two arguments indicating the values of X and Y.
- The following example plots $y = 2x^2 - 3$

```
X=[[2,3,1],[4,6,3],[6,9,7],[8,10,5],[9,11,7],[10,18,12],
[11,23,14]]
plt.plot(X)
plt.show()
```

- One can even pass a two dimensional array (or list of lists) in plot, in which case the first element of each row (or list) would be plotted as a separate plot and the second as a separate plot.

The Plot Function

```
import matplotlib.pyplot as plt
plt.plot([1,4,8,10], color='red')
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.axis([0,6,0,15])
plt.show()
plt.savefig("line.png",dpi=80)
```

- The *color* argument of the plot function can be set to a particular color of the plot. The default color is blue.
- In the following example, the x-axis would now span from 0 to 6 and y-axis would span from 0 to 15, owing to the arguments of the axis function.

```
plt.plot([1,3,4],[7,8,3],'o', color='red')
plt.plot([1,2,3,4],[2,1,3,5],'s', color='green')
plt.plot([1,5,6],[9,10,11],'^', color='black')
```

- If one wants to plot only the points and not the lines, an additional argument “o” can be passed to the plot function.
- square and a triangle can be plotted by giving “s” and “^.”

The Plot Function

- import matplotlib.pyplot as plt
- plt.plot([1,3,4],[7,8,3], color='red',linewidth=1.5, linestyle="-", label="Dash line")
- plt.plot([1,2,3,4],[2,1,3,5], color='green',linewidth=2.5, linestyle="--", label="Double Dash line")
- plt.plot([1,5,6],[9,10,11], color='black',linewidth=2.5, linestyle="-.", label="Dash Dot line")
- plt.legend(loc='upper left')
- plt.show()

- Linewidth attribute sets width of line
- Linestyle attribute set style of line like '-', '--', '-.', ':', 'None', ' ', '', 'solid', 'dashed', 'dashdot', 'dotted'.
- To add a legend in the upper left corner you will only requires adding the keyword argument label (that will be used in the legend box) to the plot commands.

The Plot Function

- import matplotlib.pyplot as plt
- plt.figure(1) # the first figure
- plt.subplot(211) # the first subplot in the first figure
- plt.plot([1, 2, 3])
- plt.subplot(212) # the second subplot in the first figure
- plt.plot([4, 5, 6])
- plt.figure(2) # a second figure
- plt.plot([4, 5, 6]) # creates a subplot(111) by default
- plt.figure(1) # figure 1 current; subplot(212) still current
- plt.subplot(211) # make subplot(211) in figure1 current
- plt.title('Easy as 1, 2, 3') # subplot 211 title
- plt.show()

- The *figure() command* is used to create new figure where our graph is display.
- The *subplot()* command specifies numrows, numcols, fignum where fignum ranges from 1 to numrows*numcols.
- The commas in the subplot command are optional if numrows*numcols<10.
- So subplot(211) is identical to subplot(2, 1, 1).
- You can create multiple figures by using multiple figure() calls with an increasing figure number.

The Scatter Plot

- Scatter plots are used to observe relationship between variables and uses dots to represent the relationship between them.
- The scatter() method in the matplotlib library is used to draw a scatter plot.
- The syntax for scatter() method is given below:
- *matplotlib.pyplot.scatter(x_axis_data, y_axis_data, s=None, c=None, marker=None, cmap=None, vmin=None, vmax=None, alpha=None, linewidths=None, edgecolors=None)*

- x_axis_data- An array containing x-axis data
- y_axis_data- An array containing y-axis data
- s- marker size (can be scalar or array of size equal to size of x or y)
- c- color of sequence of colors for markers
- marker- marker style

- cmap- cmap name
- linewidths- width of marker border
- edgecolor- marker border color
- alpha- blending value, between 0 (transparent) and 1 (opaque)

The Scatter Plot

```
# dataset-1
x1 = [89, 43, 36, 36, 95, 10, 66, 34, 38, 20]
y1 = [21, 46, 3, 35, 67, 95, 53, 72, 58, 10]
# dataset2
x2 = [26, 29, 48, 64, 6, 5, 36, 66, 72, 40]
y2 = [26, 34, 90, 33, 38, 20, 56, 2, 47, 15]
plt.scatter(x1, y1, c ="pink",linewidths = 2, marker ="s", edgecolor ="green", s = 50)
plt.plot(x1,y1)            #Connect points using line (Optional)
plt.scatter(x2, y2, c ="yellow",linewidths = 2,marker ="^",edgecolor ="red",s = 200)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend(["Dataset 1" , "Dataset 2"],ncol = 2 , loc = "lower right")
plt.show()
```

The Scatter Plot

- Example 2 # import required modules
- import matplotlib.pyplot as plt
- import numpy
- # assign data points
- a = numpy.array([[9, 1, 2, 7, 5, 8, 3, 4, 6],
- [4, 2, 3, 7, 9, 1, 6, 5, 8]])
- # assign categories
- categories = numpy.array([0, 1, 2, 0, 1, 2, 0, 1, 2])
- # use colormap
- colormap = numpy.array(['r', 'g', 'b'])
- # depict illustration
- plt.scatter(a[0], a[1], s=100, c=colormap[categories])
- plt.show()

- Example 3 # import required modules
- plt.style.use('seaborn')
- x = [1,2,3,4,5,6,7,8,9,10,11,12]
- y = [1,2,3,4,5,6,7,8,9,10,11,12]
- points_size = [100,200,300,400,500,600,700,800,900,1000,1100,1200]
- plt.scatter(x,y,s=points_size,c='g')
- plt.title("Scatter Plot with increase in size of scatter points ", fontsize=22)
- plt.xlabel('x-axis',fontsize=20)
- plt.ylabel('y-axis',fontsize=20)
-
- plt.show()

The Bar Plot

- A bar plot or bar chart is a graph that represents the category of data with rectangular bars with lengths and heights that is proportional to the values which they represent.
- The bar plots can be plotted horizontally or vertically.
- A bar chart describes the comparisons between the discrete categories.
- One of the axis of the plot represents the specific categories being compared, while the other axis represents the measured values corresponding to those categories.
- The bar() function which can be used.
- The syntax of the bar() function to be used with the axes is as follows:-
- *plt.bar(x, height, width, bottom, align)*
- *plt.barh()* is used to create horizontal bar

The Bar Plot

```
import numpy as np
import matplotlib.pyplot as plt
# creating the dataset
data = {'C':20, 'C++':15, 'Java':30,'Python':35}
courses = list(data.keys())
values = list(data.values())
fig = plt.figure(figsize = (10, 5))
# creating the bar plot
plt.bar(courses, values, color ='maroon',width = 0.4)
plt.xlabel("Courses offered")
plt.ylabel("No. of students enrolled")
plt.title("Students enrolled in different courses")
plt.show()
```

```
import numpy as np
import matplotlib.pyplot as plt
# creating the dataset
data = {'C':20, 'C++':15, 'Java':30,'Python':35}
courses = list(data.keys())
values = list(data.values())
fig = plt.figure(figsize = (10, 5))
# creating the Horizontal bar plot
plt.barh(courses, values, color ='maroon')
plt.xlabel("Courses offered")
plt.ylabel("No. of students enrolled")
plt.title("Students enrolled in different courses")
plt.show()
```

The Bar Plot

```
import numpy as np
import matplotlib.pyplot as plt
 # set width of bar
barWidth = 0.25
fig = plt.subplots(figsize =(12, 8))
 # set height of bar
IT = [12, 30, 1, 8, 22]
ECE = [28, 6, 16, 5, 10]
CSE = [29, 3, 24, 25, 17]
 # Set position of bar on X axis
br1 = np.arange(len(IT))
br2 = [x + barWidth for x in br1]
br3 = [x + barWidth for x in br2]
```

- Multiple bar example

```
# Make the plot
plt.bar(br1, IT, color ='r', width = barWidth,
        edgecolor ='grey', label ='IT')
plt.bar(br2, ECE, color ='g', width = barWidth,
        edgecolor ='grey', label ='ECE')
plt.bar(br3, CSE, color ='b', width = barWidth,
        edgecolor ='grey', label ='CSE')
 # Adding Xticks
plt.xlabel('Branch', fontweight ='bold', fontsize = 15)
plt.ylabel('Students passed', fontweight ='bold', fontsize = 15)
plt.xticks([r + barWidth for r in range(len(IT))],
        ['2015', '2016', '2017', '2018', '2019'])
 plt.legend()
plt.show()
```

The Bar Plot

```
import matplotlib.pyplot as plt
import numpy as np

# create data
x = ['A', 'B', 'C', 'D']
y1 = np.array([10, 20, 10, 30])
y2 = np.array([20, 25, 15, 25])
y3 = np.array([12, 15, 19, 6])
y4 = np.array([10, 29, 13, 19])

```

```
# plot bars in stack manner
plt.bar(x, y1, color='r')
plt.bar(x, y2, bottom=y1, color='b')
plt.bar(x, y3, bottom=y1+y2, color='y')
plt.bar(x, y4, bottom=y1+y2+y3, color='g')
plt.xlabel("Teams")
plt.ylabel("Score")
plt.legend(["Round 1", "Round 2", "Round 3", "Round 4"])
plt.title("Scores by Teams in 4 Rounds")
plt.show()
```

- Stacked bar example

The Bar Plot

```
import numpy as np
import matplotlib.pyplot as plt
 # create data
A = np.array([3,6,9,4,2,5])
B = np.array([2,8,1,9,7,3])
X = np.arange(6)
 # plot the bars
plt.barh(X, A, color = 'r', alpha=0.4)
plt.barh(X, -B, color = 'b')
plt.title("Back-to-Back Bar Chart")
plt.show()
```

- Back to Back bar example

The Bar Plot

- There are two different ways to display the values of each bar in a bar chart in matplotlib –
  - Using matplotlib.axes.Axes.text() function.
  - Use matplotlib.pyplot.text() function.
- Using matplotlib.axes.Axes.text() function.
  - This function is basically used to add some text to the location in the chart.
  - This function return string, this is always used with the syntax “for index, value in enumerate(iterable)” with iterable as the list of bar values to access each index, value pair in iterable so at it can add the text at each bar.
- Use matplotlib.pyplot.text() function.
  - Call matplotlib.pyplot.barh(x, height) with x as a list of bar names and height as a list of bar values to create a bar chart.
  - Use the syntax “for index, value in enumerate(iterable)” with iterable as the list of bar values to access each index, value pair in iterable.
  - At each iteration, call matplotlib.pyplot.text(x, y, s) with x as value, y as index, and s as str(value) to label each bar with its size.

The Bar Plot

- Using matplotlib.axes.Axes.text() function.
- # creating the dataset
- data = {'C':20, 'C++':15, 'Java':30,'Python':35}
- courses = list(data.keys())
- values = list(data.values())
- fig = plt.figure(figsize = (10, 5))
- # creating the bar plot
- plt.barh(courses, values, color ='maroon')
- plt.xlabel("Courses offered")
- plt.ylabel("No. of students enrolled")
- plt.title("Students enrolled in different courses")
- for i, v in enumerate(values):
- plt.text(v + 1, i + .25, str(v), color = 'blue', fontweight = 'bold')
- plt.show()

- Use matplotlib.pyplot.text() function.
- # creating the dataset
- data = {'C':20, 'C++':15, 'Java':30,'Python':35}
- courses = list(data.keys())
- values = list(data.values())
- fig = plt.figure(figsize = (10, 5))
- # creating the bar plot
- plt.barh(courses, values, color ='maroon')
- plt.xlabel("Courses offered")
- plt.ylabel("No. of students enrolled")
- plt.title("Students enrolled in different courses")
- for index, value in enumerate(values):
- plt.text(value, index,str(value))
- plt.show()

The Histogram

- A histogram is basically used to represent data provided in a form of some groups.It is accurate method for the graphical representation of numerical data distribution.
- It is a type of bar plot where X-axis represents the bin ranges while Y-axis gives information about frequency.
- A histogram is a graph showing frequency distributions.
- It is a graph showing the number of observations within each given interval.
- In Matplotlib, we use the hist() function to create histograms.
- The hist() function will use an array of numbers to create a histogram, the array is sent into the function as an argument.
- To create a histogram the first step is to create bin of the ranges, then distribute the whole range of the values into a series of intervals, and count the values which fall into each of the intervals.
- Bins are clearly identified as consecutive, non-overlapping intervals of variables.

The Histogram

- The towers or bars of a histogram are called bins.
- The height of each bin shows how many values from that data fall into that range.
- The default value of the number of bins to be created in a histogram is 10.
- Width of each bin is = (max value of data – min value of data) / total number of bins
- We can also pass a sequence of int or float in the parameter bins.
- In which the elements of the sequence are the edges/boundaries of the bins. In this method, the bin width may vary for each bin.
- Suppose a sequence [1,2,3,4,5] is assigned to bins and then the number of bins made will be 4 i.e the first bin will be [1,2) (including 1, but excluding 2) second bin will be [2,3) (including 2, but excluding 3) third bin will be [3,4) (including 3, but excluding 4). However, in the last bin [4,5] both 4 and 5 are included.

The Histogram

- The following table shows the parameters accepted by matplotlib.pyplot.hist() function :

Attribute	parameter
x	array or sequence of array
bins	optional parameter contains integer or sequence or strings
density	optional parameter contains boolean values
range	optional parameter represents upper and lower range of bins
histtype	optional parameter used to create type of histogram [bar, barstacked, step, stepfilled], default is “bar”
align	optional parameter controls the plotting of histogram [left, right, mid]
weights	optional parameter contains array of weights having same dimensions as x
bottom	location of the basline of each bin
rwidth	optional parameter which is relative width of the bars with respect to bin width
color	optional parameter used to set color or sequence of color specs
label	optional parameter string or sequence of string to match with multiple datasets
log	optional parameter used to set histogram axis on log scale

The Histogram

- The pyplot function can also be used to plot a histogram.
- The *hist* function can be used to accomplish the task.
- The hist function has *data* as an argument.
- The *color* argument of the function associates the color to the histogram.
- The optional *commutative* argument, if True, plots a commutative histogram.
- The number of *bins* indicates the segregations on the x-axis

```
data = np.random.randn(100)
f, (ax1, ax2) = plt.subplots(1,2,figsize=(6,3))
ax1.hist(data,bins=10,color='blue')
ax2.hist(data,bins=10,color='red',cumulative=True)
plt.show()
```

The Pie Chart

- A Pie Chart is a circular statistical plot that can display only one series of data.
- The area of the chart is the total percentage of the given data.
- The area of slices of the pie represents the percentage of the parts of the data.
- The slices of pie are called wedges.
- The area of the wedge is determined by the length of the arc of the wedge.
- The area of a wedge represents the relative percentage of that part with respect to whole data.
- Pie charts are commonly used in business presentations like sales, operations, survey results, resources, etc as they provide a quick summary.

The Pie Chart

- Matplotlib API has pie() function in its pyplot module which create a pie chart representing the data in an array.
- *Syntax: matplotlib.pyplot.pie(data, explode=None, labels=None, colors=None, autopct=None, shadow=False)*
- *data* represents the array of data values to be plotted, the fractional area of each slice is represented by data/sum(data). If sum(data)<1, then the data values returns the fractional area directly, thus resulting pie will have empty wedge of size 1-sum(data).
- *labels* is a list of sequence of strings which sets the label of each wedge.
- *color* attribute is used to provide color to the wedges.
- *autopct* is a string used to label the wedge with their numerical value.
- *shadow* is used to create shadow of wedge.

The Pie Chart

- Maybe you want one of the wedges to stand out?
- The explode parameter allows you to do that.
- The explode parameter, if specified, and not None, must be an array with one value for each wedge.
- Each value represents how far from the center each wedge is displayed:

Hexbin plot

- A hexbin plot in Matplotlib is a 2D histogram representation of data points using hexagonal bins instead of rectangular bins.
- It's useful for visualizing the distribution and density of data points.
- You can create a hexbin plot in Matplotlib using the **hexbin** function.

Syntax

```
matplotlib.pyplot.hexbin(x, y, C=None, gridsize=100, bins=None, cmap=None, norm=None, vmin=None, vmax=None,
alpha=None, linewidths=None, edgecolors='face', reduce_C_function=<function mean at 0x...>, mincnt=None,
marginals=False, data=None, **kwargs)
```

Hexbin plot

- **x**: The x-coordinates of the data points.
- **y**: The y-coordinates of the data points.
- **C** (optional): The values associated with each data point (e.g., weights or counts).
- If not specified, it will be treated as a count of 1 for each point.
- **gridsize** (optional): The number of hexagons in the x-direction. Default is 100.
- **bins** (optional): A specification of the bin edges. If specified, it takes precedence over **gridsize**.
- **cmap** (optional): The colormap for coloring the hexagons.
- **norm** (optional): Normalize data values into the [0, 1] range for colormap scaling.
- **vmin** and **vmax** (optional): The minimum and maximum values for colormap scaling.
- **alpha** (optional): The transparency of the hexagons.
- **linewidths** (optional): The width of the hexagon edges.
- **edgecolors** (optional): The color of the hexagon edges. You can set it to 'face' to use the same color as the hexagon face.
- **reduce_C_function** (optional): A function to reduce **C** values within each hexagon. Default is **np.mean**.
- **mincnt** (optional): Ignore bins with fewer than **mincnt** counts.
- **marginals** (optional): Whether to add marginal histograms (default is **False**).

The **hexbin** function returns an object that represents the hexbin plot, which you can use to customize the plot further or add a colorbar.

Hexbin plot

```
import numpy as np
import matplotlib.pyplot as plt

# Generate some random data
np.random.seed(0)
x = np.random.normal(0, 1, 1000)
y = np.random.normal(0, 1, 1000)

# Create a hexbin plot
plt.hexbin(x, y, gridsize=30,
cmap='viridis')

# Add a colorbar for reference
plt.colorbar(label='Count in Bin')

# Set labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Hexbin Plot Example')
```

In this example:

1. We generate some random data points x and y.
2. We use the plt.hexbin function to create the hexbin plot.
3. You can adjust the gridsize parameter to control the size of the hexagonal bins.
4. The cmap parameter specifies the colormap for coloring the hexagons.
3. We add a colorbar to indicate the count of points in each bin.
4. Labels and a title are set for the plot.
5. Finally, we display the plot using plt.show().

You can replace the x and y arrays with your own data to create a hexbin plot for your specific dataset.